MLOps Course Project

Design, Automate, Deploy, and Monitor an End-to-End ML System

Team size	Exactly 3 students
Project type	End-to-end data science / machine learning
Deployment requirement	Cloud deployment using AWS or an approved equivalent AWS architecture
Primary objective	Build a reproducible, automated, tested, deployed, and monitored ML system

Project specification, monitoring and retraining requirements, deliverables, assessment rubric, and acceptance criteria

1. Project Overview and Objective

Each team must select a meaningful data science or machine learning problem and implement a complete working system. The project must automate the journey from raw data through validation, preprocessing, training, evaluation, deployment, prediction, and monitoring. The objective is not merely to achieve high model accuracy; the system must demonstrate reproducibility, automation, testability, reliability, security, and practical deployment.

Possible application areas

- Healthcare risk prediction using an appropriate public dataset.
- Customer churn prediction.
- Fraud or anomaly detection.
- Crop disease classification or agricultural yield prediction.
- Sentiment analysis or text classification using Hugging Face.
- Predictive maintenance or equipment-failure prediction.
- Another application approved by the instructor.

2. Required End-to-End Architecture

The system should implement the following lifecycle. Teams should automate as many steps as practical, while reserving manual approval for appropriate production decisions.

Stage	Purpose	Suggested tools
1. Data acquisition and validation	Dataset ingestion, schema checks, quality validation, preprocessing.	Python, Pandas, Airflow, Pandera or Great Expectations (optional)
2. Version control and reproducibility	Version code, data, configurations, and pipeline definitions.	Git, GitHub, DVC, DagsHub
3. Automated training pipeline	Feature engineering, training, tuning, evaluation.	Modular Python, DVC pipelines, Airflow, Kubeflow
4. Experiment tracking and model management	Record parameters, metrics, artifacts, model versions, and approval status.	MLflow, DagsHub
5. Automated quality gates	Unit tests, data tests, model evaluation, and CI checks.	pytest, GitHub Actions
6. Containerization and deployment	Package the model API and deploy it to the cloud.	Flask, Docker, Docker Compose, AWS; Kubernetes
7. Monitoring and improvement	Track logs, latency, errors, drift, and performance; plan retraining.	CloudWatch, application metrics, suitable drift-monitoring tools

3. Mandatory Technical Requirements

3.1 Modular ML application development

Develop reusable Python modules. Do not implement the entire system in a single notebook or script.

- Data ingestion and validation.
- Data preprocessing and feature engineering.
- Model training and hyperparameter tuning.
- Model evaluation and model serialization/loading.
- Prediction and API serving.
- Logging and monitoring.

Deliverables: organized source code, dependency file (requirements.txt or equivalent), configuration files, and a README with run instructions.

3.2 Git, DVC, and DagsHub

- Create a GitHub repository and use meaningful commits.
- Use branches and pull requests for team collaboration.
- Track datasets and data artifacts with DVC.
- Configure a DVC remote, such as DagsHub storage or an AWS storage bucket.
- Record the data and code versions associated with each experiment.
- Demonstrate retrieving an earlier code version and its corresponding dataset.
- Document handling of merge conflicts, including conflicts involving .dvc files.

Deliverables: GitHub repository, DVC configuration, version history, and reproducibility demonstration.

3.3 MLflow experiment tracking

Use MLflow to track experiments. Each run should record, where applicable:

- Model type and algorithm.
- Training parameters and hyperparameters.
- Dataset or DVC version.
- Evaluation metrics and training duration.
- Model artifacts, plots, and relevant outputs.
- Code version or Git commit identifier.

Compare multiple experiments and justify the selected model. Use an MLflow Model Registry or the model-management functionality supported by the team's setup to identify model versions and statuses.

3.4 Automated Data Science & ML Pipeline (Dual Orchestration: Airflow + Kubeflow)

Implement an automated end-to-end pipeline leveraging the dual-orchestration architecture where Airflow manages scheduling/data ingestion, and Kubeflow Pipelines manages the ML workflow graph:

1. **Airflow Scheduled Ingestion & Validation:**
    - Apache Airflow runs on a schedule (or external webhook) to fetch raw data from source systems or AWS S3.
    - Airflow tasks execute data quality and schema validation checks (e.g., missing values, type consistency).
2. **Kubeflow Pipeline Trigger & Execution:**
    - Upon successful validation, Airflow programmatically triggers the Kubeflow Pipeline (via API or CLI).

- Kubeflow Pipelines executes the component-based ML graph on a Kubernetes environment (managed cloud cluster or local Minikube/Kind for budget efficiency):
  - ✧ Step A: Feature engineering and data splitting (train/val/test).
  - ✧ Step B: Model training and hyperparameter tuning across containerized pods.
  - ✧ Step C: Model evaluation against threshold criteria and automatic logging to MLflow.
  - ✧ Step D: Model packaging, container build, and registration into the Model Registry.

3. End-to-End Automation:

The entire pipeline must run end-to-end via a single Airflow DAG trigger or automated workflow invocation, eliminating manual notebook cell execution or manual script calls.

3.5 Automated testing

Test category	Examples
Unit tests	Validate preprocessing functions and prediction logic.
Data tests	Check schema, missing values, data types, and invalid records.
Model tests	Verify output shape, valid predictions, and acceptable metrics.
API tests	Test valid requests, malformed input, and error responses.
Integration tests	Verify preprocessing, model loading, and prediction work together.
Regression tests	Detect unexpected deterioration in model quality.

Use pytest or an equivalent framework. Define a minimum model-performance threshold suitable for the problem. A model that fails mandatory quality checks must not automatically proceed to deployment.

3.6 GitHub Actions and CI/CD

- Trigger workflows on code pushes and pull requests.
- Install dependencies and run linting/code-quality checks.
- Execute automated tests and build the Docker image.
- Block progress when mandatory quality gates fail.
- Publish the image to a container registry after required checks pass.
- Deploy approved versions to the target environment.
- Use GitHub secrets or identity-based authentication for cloud access; never hard-code credentials.

Deliverables: working workflow files, execution logs, and evidence of successful and failed quality gates.

3.7 Docker and Dockerfiles

- Create a Dockerfile that installs dependencies and copies the required source and model artifacts.
- Configure the startup command and expose the required API port.
- Use a suitable production server for Flask.
- Avoid embedding secrets or unnecessary datasets in the image.
- Demonstrate consistent execution in a container on another machine.
- Use Docker Compose for the API and any required supporting services, such as MLflow or monitoring components.

- Keep development and production configurations appropriately separated.

3.8 Flask REST API and Hugging Face

Expose the trained model through a Flask-based REST API with the following endpoints:

Endpoint	Requirement
GET /health	Report application health.
GET /model-info	Return model version and relevant metadata.
POST /predict	Accept input data and return predictions.

Teams using Hugging Face must integrate a pretrained or fine-tuned model into the serving architecture where appropriate. For example, sentiment analysis may use a transformer, while a tabular problem may use scikit-learn. The API must validate requests, handle errors, return appropriate HTTP status codes, and avoid unnecessarily logging sensitive inputs.

Deliverables: API source code, request/response examples, automated API tests, and a demonstration using curl, Postman, or an equivalent client.

3.9 AWS cloud deployment

Capability	Suggested AWS service
Container image storage	Amazon ECR
Model/API hosting	Amazon ECS with Fargate, or EC2
Model and dataset artifact storage	Amazon S3
Application logs and monitoring	Amazon CloudWatch
Credentials and configuration	IAM roles and, where appropriate, Secrets Manager
Automated deployment	GitHub Actions with AWS authentication

Teams may use Amazon EKS for Kubernetes deployment if infrastructure and cost permit. AWS Lambda or SageMaker may be used as alternatives for suitable architectures, but teams must still demonstrate the required containerization, API serving, and deployment workflow. Local execution alone is insufficient.

The deployed service must have a documented endpoint and appropriate access controls.

AWS cost and safety requirements

- Use the smallest practical resources.
- Set a budget alert before deployment.
- Do not expose cloud credentials in GitHub.
- Restrict inbound traffic and permissions.
- Stop or delete billable resources when demonstrations are complete.
- Document the services used and steps required to reproduce deployment.

3.10 Kubernetes scaling

- Deploy the containerized inference API using a Kubernetes Deployment and Service.
- Use a local environment such as Minikube or kind if needed.
- Configure readiness and liveness probes.
- Scale the application to multiple replicas.
- Demonstrate a failed container being restarted or replaced.
- Explain load balancing and remaining limitations.

Docker Compose is required for multi-container development or local integration. Kubernetes must be used to demonstrate orchestration and scaling; a second production Kubernetes cluster on AWS is not mandatory if it creates excessive cost.

3.11 Airflow and Kubeflow orchestration

Airflow: Create a DAG with task dependencies for data validation, preprocessing, training, evaluation, and subsequent pipeline actions. Configure retries and logging and demonstrate manual or scheduled execution.

Kubeflow Pipelines: Build or adapt a pipeline expressing the major ML stages as components with defined inputs and outputs. Demonstrate an execution and explain how component outputs feed later stages.

The tools need not run duplicate workloads continuously. Explain their respective roles and demonstrate the required workflows in a practical, cost-conscious environment.

3.12 Monitoring and feedback

- API request count and error rate.
- Prediction latency and application health.
- Model version used for each prediction.
- Data-quality failures.
- Input data drift or changes in input distributions.
- Model performance when ground-truth labels become available.

Use Prometheus to collect operational metrics and Grafana to visualize them and configure alerts. Use CloudWatch for AWS infrastructure metrics and application logs, supplemented by an ML monitoring library or custom monitoring code. Define at least one alert or threshold (for example, excessive API errors, increased latency, or input drift). Explain how the issue would be investigated and how retraining could be initiated.

Deliverables: Grafana dashboard screenshots or a live demonstration, monitoring configuration, alert evidence, Airflow retraining workflow, MLflow experiment records, and documentation of model promotion and rollback.

Teams must demonstrate a monitoring dashboard, a configured alert, a retraining workflow, candidate-model evaluation, and the deployment decision. If real production drift or new labels are unavailable, a controlled test scenario or simulated drift event may be used.

C. Demonstration and deliverables

- Deploy an approved candidate through CI/CD and demonstrate how to roll back to the previous version.
- Prevent deployment if the candidate fails required quality or safety checks.
- Compare the candidate model against the currently deployed model using predefined evaluation criteria.
- Log experiments, metrics, and model artifacts in MLflow.
- Revalidate the data and retrain the model using the established ML pipeline.

- Support scheduled retraining and at least one defined trigger, such as significant data drift or a decline in model performance.
- Implement a retraining workflow orchestrated by Airflow.

B. Automated retraining

- Track the deployed model version and evaluate predictive performance when ground-truth labels become available.
- Implement data-quality and data-drift checks using Evidently or an equivalent approach.
- Use AWS CloudWatch for appropriate AWS infrastructure metrics and application logs.
- Configure at least one alert based on a defined operational threshold, such as excessive errors or elevated latency.
- Configure Grafana dashboards to visualize the collected metrics and relevant resource utilization.
- Integrate Prometheus to collect relevant application metrics, including API request rate, error rate, prediction latency, and application health.

A. Monitoring and dashboards

This is a mandatory extension to the monitoring and feedback requirements above. Each team must demonstrate operational observability and a controlled model-retraining lifecycle. Grafana and Prometheus complement rather than replace MLflow, Evidently, Airflow, or CloudWatch.

3.12A Grafana, Prometheus, and Automated Model Retraining

3.13 Security, privacy, and ethics

- Securely handle API keys, passwords, and cloud credentials.
- Apply least-privilege IAM permissions and access controls.
- Protect sensitive data and remove or control sensitive attributes as appropriate.
- Validate inputs and consider malformed or abusive requests.
- Check dataset licensing and permitted use.
- Discuss model limitations, bias, fairness, explainability, and potential harm.
- Maintain reproducibility and auditability of model versions.
- Use safe logging and appropriate data-retention practices.

Include a risk assessment with at least three project-specific risks and mitigation strategies.

4. Team Organization and Responsibilities

Each group must consist of exactly three students. The following can be primary responsibilities, not isolated work areas. All members must understand the overall architecture, review one another's code, participate in integration, and be ready to explain or execute any component during the oral defense. However, it is optional, any student can work on any module.

Role	Primary responsibilities
Student 1 — Data and ML Engineering	Dataset selection; data cleaning, validation, and feature engineering; modular ML code; training and evaluation; DVC data versioning; MLflow experiment tracking.
Student 2 — MLOps and	Git/GitHub collaboration; DagsHub integration; DVC pipeline; Airflow DAGs;

Automation Engineering	Kubeflow Pipelines; automated tests; GitHub Actions and CI/CD.
Student 3 — Deployment and Reliability Engineering	Flask REST API; Hugging Face integration where appropriate; Docker and Docker Compose; Kubernetes; AWS deployment; access control, monitoring, and logging.

5. Recommended Repository Structure

```
mlops-course-project/
├── data/
│   ├── raw/                          # Raw datasets (tracked by DVC)
│   └── processed/                    # Feature engineered outputs
├── src/
│   ├── data/
│   │   ├── ingestion.py              # Data retrieval scripts
│   │   └── validation.py             # Schema & data quality checks
│   ├── features/
│   │   └── preprocessing.py          # Feature transformation modules
│   ├── models/
│   │   ├── train.py                  # Training logic
│   │   ├── tune.py                   # Hyperparameter tuning logic
│   │   └── evaluate.py               # Evaluation metrics & thresholds
│   ├── serving/
│   │   └── app.py                    # FastAPI REST application
│   └── monitoring/
│       └── monitor.py                # Data drift / API performance monitoring
├── tests/
│   ├── test_data.py                  # Pytest data checks
│   ├── test_model.py                 # Pytest model quality checks
│   ├── test_api.py                   # Pytest REST endpoint tests
│   └── test_pipeline.py              # Integration tests
├── pipelines/
│   ├── airflow/
│   │   └── dags/
│   │       └── ml_orchestrator_dag.py  # Airflow DAG (Ingestion + KFP Trigger)
│   └── kubeflow/
│       ├── components/               # Reusable KFP component definitions
│       │   ├── preprocess_component.py
│       │   ├── train_component.py
│       │   └── evaluate_component.py
│       ├── pipeline_definition.py    # KFP SDK Python DAG definition
│       └── compiled_pipeline.yaml    # Compiled Kubeflow Pipeline IR YAML
├── configs/
│   ├── model_params.yaml             # Model hyperparameters
│   └── pipeline_config.yaml          # Infrastructure & pipeline configs
├── docker/
│   ├── Dockerfile.api                # Container build for FastAPI service
│   └── Dockerfile.ml                 # Base container build for KFP pods
├── .github/
│   └── workflows/
│       ├── ci.yml                    # Linting, Pytest, Docker build checks
│       └── deploy.yml                # Automated AWS cloud deployment
```

```
├── docker-compose.yml                    # Local multi-container setup (API + MLflow + DB)
├── k8s/
│   ├── api-deployment.yaml               # Kubernetes deployment spec for API
│   ├── api-service.yaml                  # Kubernetes service spec
│   └── aws-iam-role-policy.yaml          # AWS EKS/S3 permissions manifest
├── dvc.yaml                              # DVC data pipeline definitions
├── requirements.txt                      # Python project dependencies
├── .dvc/                                 # DVC configuration & remote tracking
├── .gitignore
├── README.md                             # Main documentation & reproduction guide
└── REPORT.md                             # Architectural walkthrough & results report
```

Adapt the structure to the selected problem. Include any required AWS configuration templates, evaluation reports, diagrams, and deployment instructions. Never commit credentials, generated secrets, or large raw datasets unnecessarily.

6. Required Automation Demonstration

1. A developer pushes code to GitHub; a pull request triggers CI checks.
2. GitHub Actions runs tests. Invalid code or failing tests block the workflow.
3. The pipeline validates and processes data; DVC tracks relevant data artifacts and versions.
4. Training and tuning run; MLflow records parameters, metrics, artifacts, and model versions.
5. The candidate model passes evaluation gates; only eligible models proceed to deployment.
6. Docker builds the image and publishes it to Amazon ECR after the required checks pass.
7. GitHub Actions deploys the approved application to AWS.
8. The service is monitored for logs, errors, latency, and data-quality indicators.
9. A scheduled or manually triggered retraining workflow runs when appropriate; a new candidate is evaluated before replacing the current model.

Teams must distinguish fully automated steps from manually triggered steps and approval gates. An automated deployment must not blindly promote every newly trained model.

7. Project Deliverables

- ☐ GitHub repository with modular code and meaningful commit history.
- ☐ DVC configuration, data version history, and reproducibility demonstration.
- ☐ DagsHub integration and MLflow experiment-tracking evidence.
- ☐ Automated training pipeline and Airflow DAG.
- ☐ Kubeflow Pipeline definition and execution evidence.
- ☐ Unit, data, model, API, and integration tests.
- ☐ GitHub Actions CI/CD workflow files and execution evidence.
- ☐ Dockerfile, Docker Compose configuration, and working container.
- ☐ Flask REST API and API documentation with example requests.
- ☐ AWS deployment evidence, endpoint details, and cost-control documentation.
- ☐ Kubernetes manifests and scaling demonstration.
- ☐ Monitoring evidence, alert threshold, and retraining plan.

- ☐ Architecture diagram and workflow diagram.
- ☐ Final technical report and individual contribution statement.
- ☐ Live demonstration and oral defense.

8. Assessment Rubric

Assessment component	Marks
Problem definition, dataset, and modular ML implementation	6
Git, DVC, DagsHub, and reproducibility	9
MLflow experiment tracking, hyperparameter tuning, model evaluation, and model registry	9
Feature Store implementation using Feast, including feature definitions, offline/online stores where appropriate, feature retrieval, and feature quality validation	6
Automated training pipelines using Apache Airflow and Kubeflow Pipelines, including scheduling, orchestration, retries, and conditional execution	12
Automated testing and CI/CD using pytest, GitHub Actions, code quality checks, and security scanning	12
Containerization and model serving using Docker, Flask REST API and BentoML, with Hugging Face integration where appropriate	10
AWS cloud deployment, including appropriate use of S3, ECR, EC2 or ECS/Fargate, CloudWatch, and related AWS services	12
Kubernetes orchestration, deployment, service configuration, scaling, and health checks	6
Monitoring, logging, drift detection, feedback collection, and automated retraining strategy	6
Security, privacy, ethics, secrets management, access control, and vulnerability scanning	2
Documentation, live demonstration, and oral defense	10
TOTAL	100

9. Minimum Acceptance Criteria

A project is considered complete only if the team demonstrates all of the following:

- The ML model can be trained using a reproducible pipeline.
- Data and code versions can be identified and recovered.
- ML experiments and evaluation metrics are recorded in MLflow.
- Automated tests run through GitHub Actions.
- A quality-gate failure prevents deployment.
- The model is served through a working Flask REST API in Docker.
- The application is deployed to AWS and can serve a valid prediction request.
- Kubernetes deployment and scaling are demonstrated.
- Airflow and Kubeflow pipeline implementations are documented and demonstrated.
- Grafana dashboards and Prometheus metrics are working; at least one alert is demonstrated; retraining is triggered by a schedule or defined condition; candidate evaluation, controlled promotion, and rollback are demonstrated; security measures are implemented, not merely described.
- Another team member or instructor can follow the README to reproduce the main workflow.

10. Recommended Industry Extensions and Scope Guidance

The following practices can improve the project where feasible. They are extensions unless the instructor explicitly makes them mandatory.

Extension	Possible tool or approach
Data validation	Great Expectations or Pandera
Model monitoring	Evidently or a lightweight custom drift detector
Code quality	Ruff and pre-commit
Model governance	Approval criteria, versioned artifacts, and rollback procedures
Infrastructure as Code	Terraform or AWS CloudFormation
Security scanning	Container image scanning and dependency vulnerability checks
Deployment strategies	Rolling deployment or a simple canary release
Reproducible configuration	Environment-specific configuration files and pinned dependencies
Operational metrics and dashboards	Prometheus and Grafana (mandatory for this project)

To keep the workload realistic, require all course tools to be addressed, but allow selected infrastructure to run locally instead of requiring every component to remain hosted on AWS. The core requirement is one complete, working vertical slice from a versioned dataset to an AWS-hosted prediction API, with automation, orchestration, scaling, monitoring, and governance demonstrated around it.

11. Submission and Demonstration Notes

- Submit the repository URL, final report, architecture diagrams, and evidence of pipeline/test/deployment execution.
- Provide setup instructions, required environment variables (names only, never secret values), and cleanup instructions.
- Include example API requests and responses, model evaluation results, and monitoring evidence.
- Identify each student's contribution and ensure all members can explain the end-to-end workflow.
- Remove or stop billable cloud resources after the demonstration unless continued hosting is explicitly approved.

12. Additional MLOps Components

Purpose. This addendum extends the specifications and adds a feature store and clarifies feedback-loop and model-serving options.

Reference: https://medium.com/@nedwinvivek/end-to-end-mlops-project-with-open-source-tools-78115cc59748

Scope note. Feast is added as the recommended feature-store implementation. BentoML is an alternative serving path; because the existing project specification explicitly requires a Flask REST API, teams should keep Flask as the required API. Moreover, the students should also demonstrate how to use BentoML. BentoML is an open-source unified inference platform that lets you package, serve, and scale AI/ML models.

12.1 Feature Store — Feast

Add a feature-store layer between feature engineering and model training/serving. Use Feast to define, version, retrieve, and document model features. For a manageable course implementation, Feast may use a local/file-based offline store and a lightweight online store during development; explain the production storage choice.

- Create a feature repository with feature definitions, entities, feature views, source definitions, and a feature_store.yaml configuration.
- Define a stable entity key and event timestamp where appropriate. Document feature names, data types, source, transformation logic, freshness expectations, and ownership.
- Load historical features for training from the offline store and retrieve low-latency features from the online store for inference where the use case requires online predictions.
- Demonstrate materializing features from the offline store to the online store and querying the online store successfully.
- Keep training and serving feature transformations consistent; test for training-serving skew, missing/stale features, and invalid feature values.
- Track feature definitions and source data with Git; version the underlying datasets with DVC. Record the feature-repository commit/version and dataset version alongside each MLflow run.
- Add tests for feature schemas, entity keys, event timestamps, freshness, nulls, and offline/online retrieval. Avoid putting secrets or sensitive raw data in the feature repository.
- For batch-only use cases, document why online retrieval is unnecessary and demonstrate the offline feature retrieval path instead.

Suggested files: feature_store/feature_repo/feature_store.yaml, feature_store/feature_repo/entities.py, feature_store/feature_repo/features.py, feature_store/feature_repo/feature_views.py, and feature_store/materialize.py (adapt names to the chosen Feast version).

12.2 Model Serving — BentoML

The team should also use BentoML to package and serve the selected model.

- Package the selected model and its required preprocessing/feature retrieval logic as a versioned serving artifact.
- Expose prediction and health endpoints, validate request schemas, return meaningful HTTP errors, and include model/version metadata.

- Build and run the serving artifact in Docker; test it with automated API and integration tests.
- Document how the model is fetched from MLflow/model registry and how a known approved model version is selected.
- Demonstrate a repeatable local smoke test and explain how the image would be promoted through the existing CI/CD workflow.

12.3 Repository Structure Additions

```
feature_store/
  feature_repo/
    feature_store.yaml
    entities.py
    features.py
    feature_views.py
  materialize.py
serving/
  app.py                  # Flask API required by the base specification
  bentoml_service.py      #  BentoML path
  model_loader.py
tests/
  test_features.py
  test_feature_store.py
```

High Level Architecture:

